

## 1. TITLE OF THE EXPERIMENT

Design and Implementation of a Dynamic Student Profile Generator Using JavaScript Classes, Event Handling and DOM Manipulation with CSS3 Styling

## 2. AIM

To design and implement a web page with a styled input form that captures student details, define a JavaScript class to model a Student object, and use event handling and dynamic DOM manipulation to generate and display a formatted profile card on the page without reloading it.

## 3. ALGORITHM / DESIGN

- Step 1:** Create the basic HTML document skeleton using `<html>`, `<head>`, `<title>` and `<body>` tags, with an internal `<style>` block for CSS.
- Step 2:** Design a centered “form-box” using CSS flexbox on the `<body>` and styled `<input>` fields for Name, Roll Number, Department and CGPA.
- Step 3:** Add a “Generate Profile” `<button>` and an empty `<div id="container">` to hold the generated profile card.
- Step 4:** Define a JavaScript class `Student` with a constructor that stores name, rollNo, department and cgpa.
- Step 5:** Attach an onclick event handler to the button that reads values from the input fields using `document.getElementById().value`.
- Step 6:** Instantiate a new `Student` object from the form values when the button is clicked.
- Step 7:** Dynamically create new DOM elements (`<div class="card">`, `<h3>`, and a details `<div>`) using `document.createElement()`.
- Step 8:** Populate the details div using a CSS Grid layout (profile-info) and template literals to align labels and values.
- Step 9:** Append the created elements to the container using `appendChild()` so the profile card renders on the page.
- Step 10:** Style the form box and profile card with CSS (rounded corners, shadows, colors) for a polished look.
- Step 11:** Save the file with a .html extension, execute it in a browser, and verify the profile card generates correctly for different inputs.

## 4. TEST CASES & EXECUTION DATA

| TC No. | Feature / Tag Tested         | Expected Result                                                                                                           | Status |
|--------|------------------------------|---------------------------------------------------------------------------------------------------------------------------|--------|
| TC-01  | Form box centering (flexbox) | The white form box appears centered both horizontally and vertically on the page                                          | Passed |
| TC-02  | Input field capture          | Name, Roll Number, Department and CGPA values are correctly read via <code>getElementById().value</code>                  | Passed |
| TC-03  | Student class instantiation  | A new <code>Student</code> object stores the four entered values in its properties                                        | Passed |
| TC-04  | Button click event handling  | Clicking “Generate Profile” triggers the onclick handler and runs the profile logic                                       | Passed |
| TC-05  | Dynamic DOM element creation | A new card <code>&lt;div&gt;</code> and <code>&lt;h3&gt;</code> title are created and appended to <code>#container</code> | Passed |
| TC-06  | Grid-aligned profile details | Labels and values (Name, Roll No, Department, CGPA) align in two neat columns                                             | Passed |

| TC No. | Feature / Tag Tested                  | Expected Result                                                                         | Status |
|--------|---------------------------------------|-----------------------------------------------------------------------------------------|--------|
| TC-07  | Repeated generation (innerHTML reset) | Clicking the button again clears the old card and displays a fresh profile              | Passed |
| TC-08  | Card styling                          | Generated card shows dark background, green heading underline and box shadow as per CSS | Passed |

## 5. LEARNING OUTCOME / RESULT

Successfully designed and implemented a Student Profile Generator that combines a styled input form with a JavaScript Student class, event-driven logic and dynamic DOM manipulation to build and display a profile card at runtime. The output was verified in a web browser for multiple inputs and matched the expected rendering for all test cases. This experiment strengthened the understanding of JavaScript classes, event handling, and creating/appending DOM elements dynamically alongside CSS Grid and Flexbox layout techniques.

## 6. PROGRAM / SOURCE CODE

File name: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Profile Generator</title>
  <style>
    /* Center everything on the screen */
    body {
      font-family: Arial, sans-serif;
      background: #eef2f3;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
    }
    /* Form Box - Enlarged */
    .form-box {
      background: #ffffff;
      padding: 35px;
      border-radius: 12px;
      box-shadow: 0 6px 15px rgba(0,0,0,0.15);
      text-align: center;
      width: 380px;
    }
    h2 {
      margin-top: 0;
      font-size: 26px;
      color: #333;
    }
    input, button {
      display: block;
      margin: 12px auto;
      padding: 12px;
      width: 90%;
      font-size: 16px;
      border-radius: 6px;
      border: 1px solid #ccc;
      box-sizing: border-box;
    }
    button {
      background: #4CAF50;
      color: white;
      border: none;
      font-weight: bold;
      cursor: pointer;
    }
    /* Generated Profile Card - Enlarged & Colorful */
    .card {
      background: #2c3e50;
      color: white;
      padding: 25px;
      margin-top: 20px;
      border-radius: 10px;
      box-shadow: 0 5px 12px rgba(0,0,0,0.2);
    }
    .card h3 {
      margin-top: 0;
      border-bottom: 2px solid #4CAF50;
      padding-bottom: 8px;
      font-size: 22px;
      color: #4CAF50;
    }
    /* CSS Grid forces perfect left and right alignment */
    .profile-info {
      display: grid;
      grid-template-columns: 110px auto;
      row-gap: 10px;
      text-align: left;
      font-size: 17px;
      font-weight: 500;
    }
  </style>
</head>
<body>
  <div class="form-box">
    <h2>Student Profile</h2>
```

```

<input type="text" id="name" placeholder="Name">
<input type="text" id="roll" placeholder="Roll Number">
<input type="text" id="dept" placeholder="Department">
<input type="number" step="0.01" id="cgpa" placeholder="CGPA">
<button id="btn">Generate Profile</button>
<div id="container"></div>
</div>
<script>
// 1. JavaScript Class
class Student {
  constructor(name, rollNo, department, cgpa) {
    this.name = name;
    this.rollNo = rollNo;
    this.department = department;
    this.cgpa = cgpa;
  }
}
// 2. Event Handling & DOM Manipulation
document.getElementById('btn').onclick = function() {
  // Create Student Object
  const s = new Student(
    document.getElementById('name').value,
    document.getElementById('roll').value,
    document.getElementById('dept').value,
    document.getElementById('cgpa').value
  );
  // Select container and clear previous content
  const container = document.getElementById('container');
  container.innerHTML = '';
  // Dynamic Element Creation
  const card = document.createElement('div');
  card.className = 'card';
  const title = document.createElement('h3');
  title.innerText = 'Student Profile';
  // Grid container for aligned details
  const infoDiv = document.createElement('div');
  infoDiv.className = 'profile-info';

  infoDiv.innerHTML = `
    <span>Name</span><span>: ${s.name}</span>
    <span>Roll No</span><span>: ${s.rollNo}</span>
    <span>Department</span><span>: ${s.department}</span>
    <span>CGPA</span><span>: ${s.cgpa}</span>
  `;
  // Append elements to DOM
  card.appendChild(title);
  card.appendChild(infoDiv);
  container.appendChild(card);
};
</script>
</body>
</html>

```